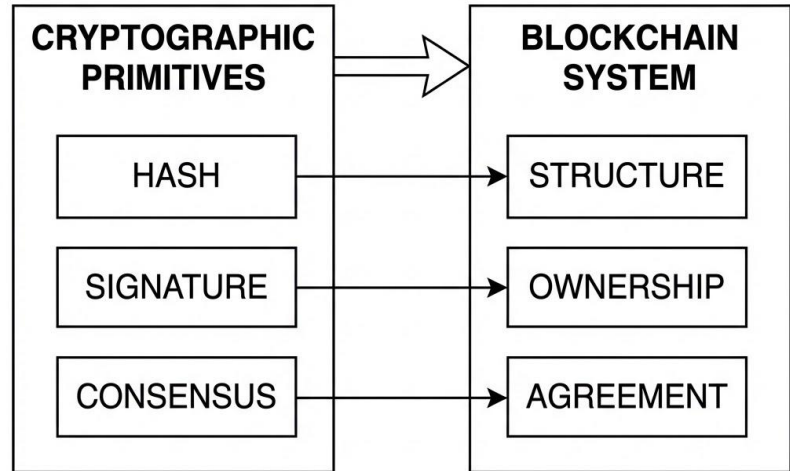


Data Structures, Wallets & Consensus in Blockchain

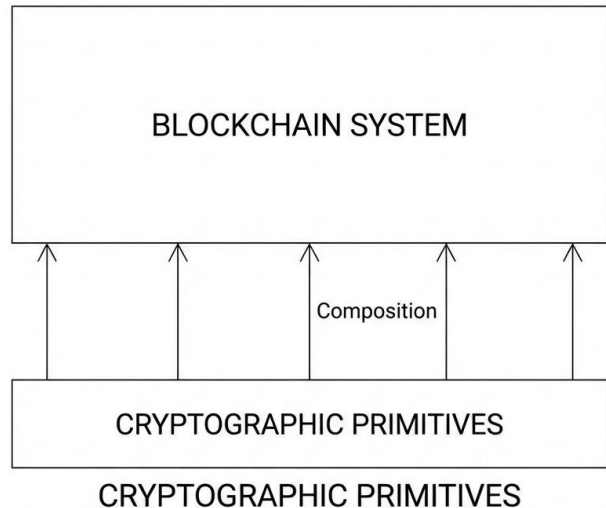
Basic Building Blocks and Beyond
Prof. Jean Martina

Data Structures, Wallets & Consensus in Blockchain

- From Cryptographic Primitives to Real Systems
- Blockchain is not a single technology, but a composition of primitives
- Today: how primitives become systems
- Focus:
 - Data structures → integrity and ordering
 - Wallets → ownership and control
 - Consensus → agreement on reality



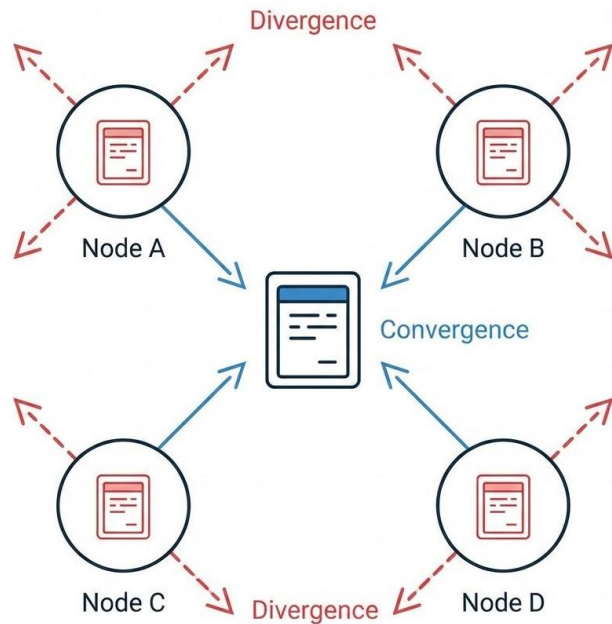
From Primitives to Systems



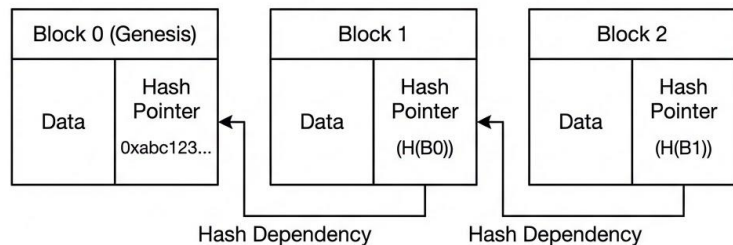
- Previous lecture:
 - Hash → integrity
 - Signatures → ownership
 - Randomness → security
 - Consensus → agreement
- Today:
 - How these primitives compose into a working system
- Key idea:
 - Blockchain = system design, not crypto invention
- Key insight: Security is emergent from composition

What Problem Are We Solving?

- Distributed system with:
 - No central authority
 - Adversarial participants
- Requirements:
 - Consistent history
 - No double spending
 - Verifiable ownership
- Core challenge
 - Multiple copies of truth → need convergence



Blockchain as a Data Structure

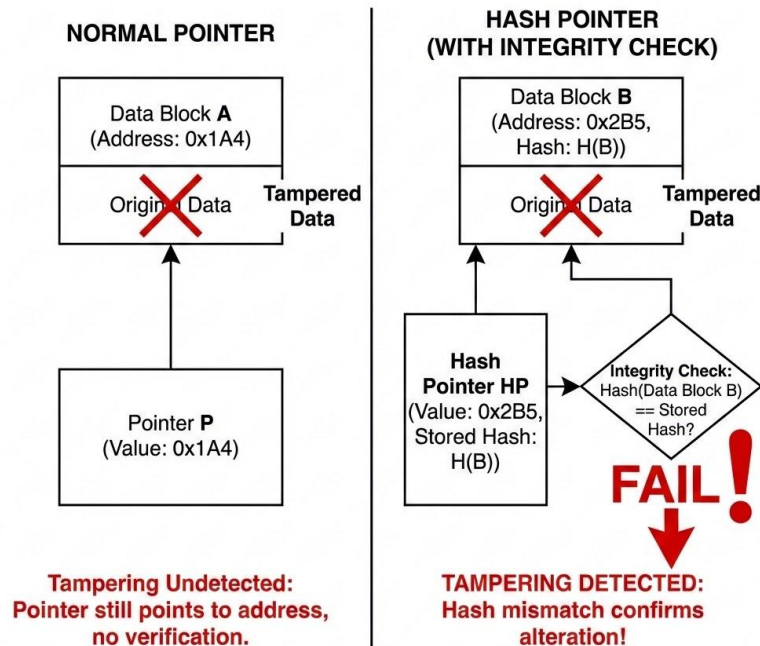


- Blockchain is:
 - A linked list
 - With hash pointers
 - With economic constraints
- Key properties:
 - Append-only
 - Immutable (practically)
 - Globally verifiable
- Important
 - Immutability is NOT absolute → it is economic

Hash Pointers vs Pointers

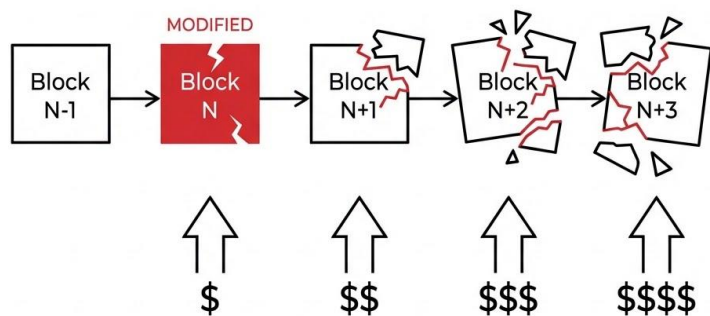
- Pointer:
 - Points to memory location
- Hash pointer:
 - Points + includes hash of content
- Consequence
 - Any change breaks the chain
- System implication
 - Integrity becomes global

Comparison: Normal Pointer vs. Hash Pointer for Data Integrity



Immutability: Economic, Not Absolute

CASCADING BLOCKCHAIN BREAK DUE TO MODIFICATION

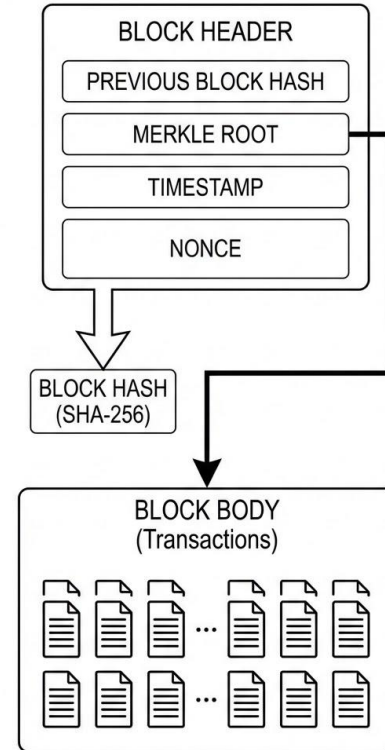


INCREASING COST OF ATTACK

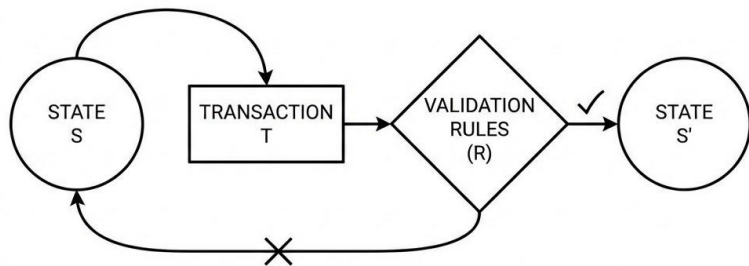
- Blockchain is often described as “immutable”
- In practice:
 - data can be changed
 - but requires recomputing all subsequent blocks
- Each modification breaks:
 - hash pointers
 - chain consistency
- To rewrite history, an attacker must:
 - recompute hashes
 - redo consensus work (e.g., PoW or PoS)
- Result:
 - cost grows with chain length
- Key insight:
 - immutability is enforced by cost, not impossibility

Block Structure: Linking Data and Integrity

- Each block contains two main components:
 - header (metadata)
 - body (transactions)
- Block header typically includes:
 - previous block hash
 - Timestamp
 - nonce (for consensus)
 - Merkle root (summary of transactions)
- The header is what gets hashed:
 - defines the identity of the block
- The body is indirectly protected:
 - via the Merkle root



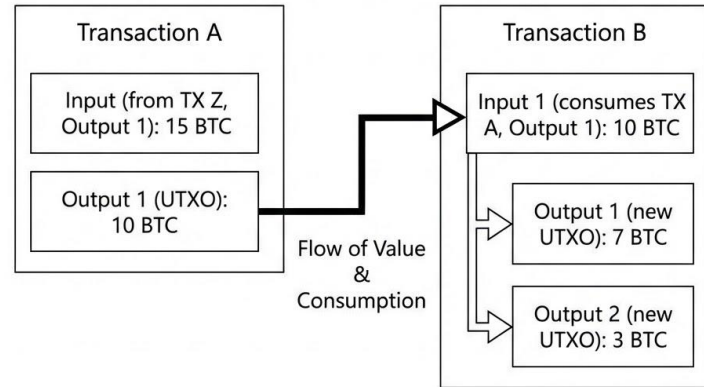
Transactions as State Transitions



- A blockchain can be modeled as a state machine
- Each transaction:
 - takes current state S
 - produces new state S'
- State includes:
 - balances (account model)
 - UTXO set (UTXO model)
- Transactions are not just records:
 - they define valid transformations
- Validation ensures:
 - rules are respected
 - state remains consistent
- Key insight:
 - blockchain is computation over state, not just storage

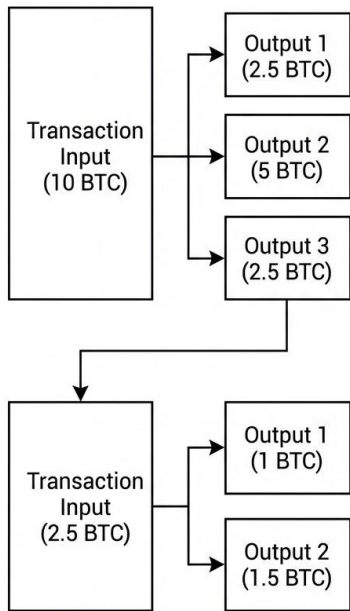
UTXO Model: Value as Spendable Outputs

- Value is represented as discrete outputs (UTXOs)
- A transaction:
 - consumes previous outputs (inputs)
 - creates new outputs
- Each input must:
 - reference a valid UTXO
 - include a valid signature
- No global balance:
 - balance = sum of controlled UTXOs
- Prevents double spending:
 - each UTXO can be spent only once

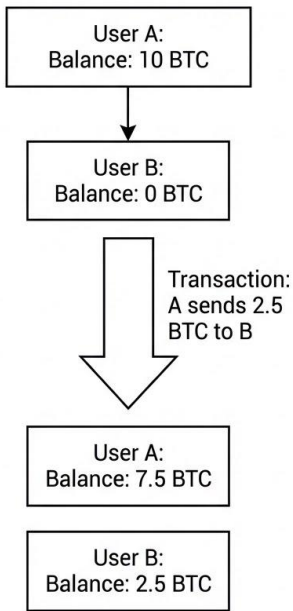


Account Model: Global State

UTXO MODEL
(FRAGMENTED OUTPUTS)



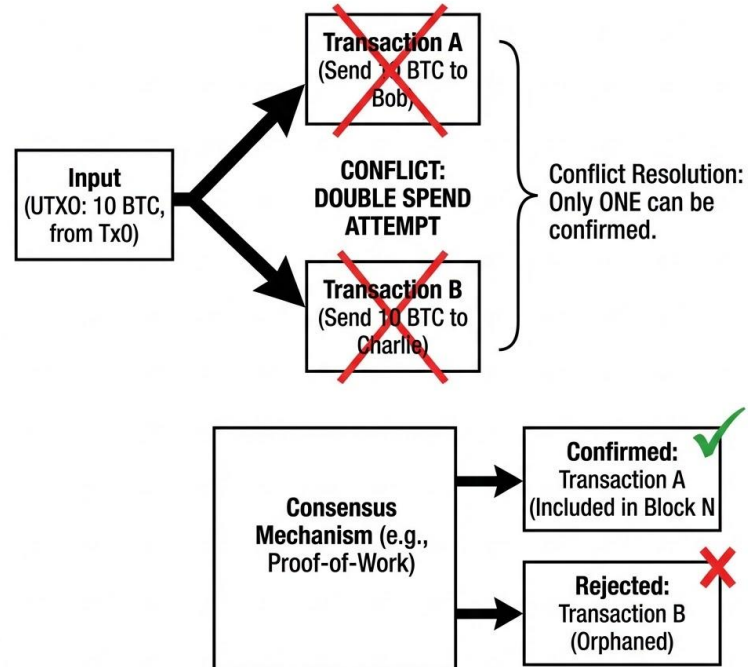
ACCOUNT BALANCE MODEL
(STATE BASED)



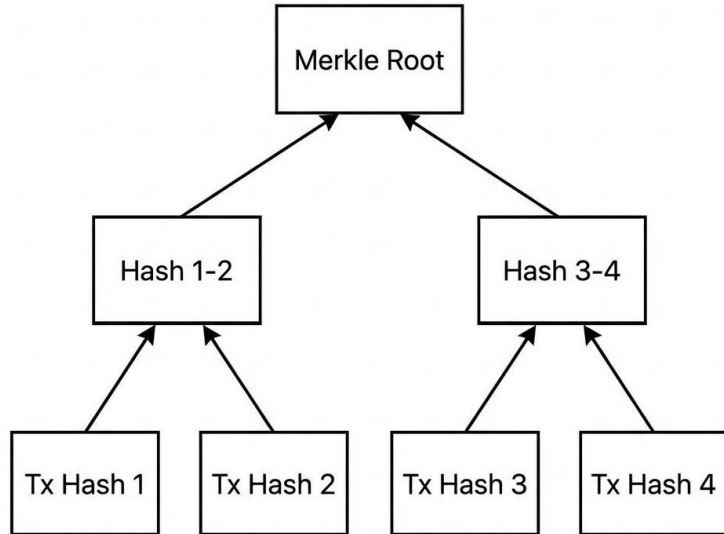
- System maintains mapping:
 - address → balance
- Transactions directly update balances
- Includes:
 - nonce (prevents replay)
 - optional smart contract logic
- Easier to understand
- Harder to parallelize
- Tradeoff:
 - simplicity vs scalability

Double Spending

- Same input used in multiple transactions
- Different nodes may see different versions
- Without resolution:
 - system becomes inconsistent
- Solution:
 - define a single valid history
- Requires:
 - ordering + consensus



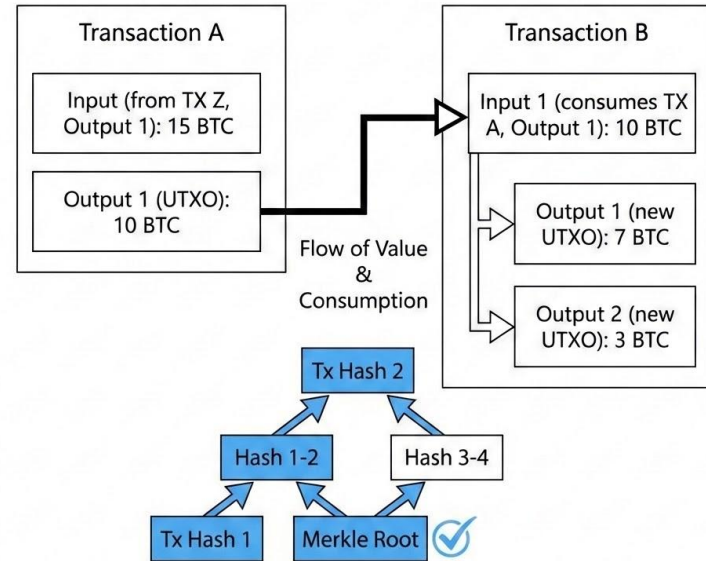
Merkle Trees



- Tree of hashes:
 - leaves = transactions
 - nodes = hash of children
- Root summarizes all data
- Any change → new root
- Used in block headers
- Benefit:
 - efficient verification

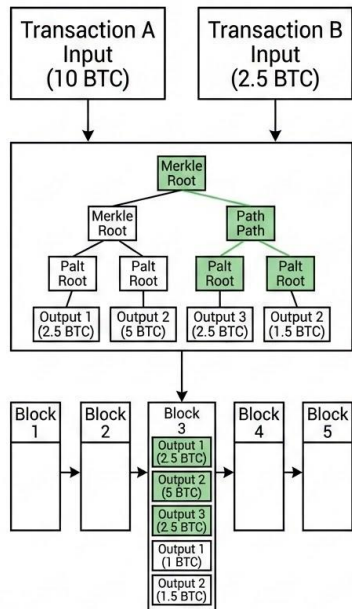
Merkle Proofs

- Prove inclusion without full data
- Requires:
 - transaction hash
 - sibling hashes along path
- Complexity:
 - $O(\log n)$
- Enables:
 - light clients
 - efficient validation

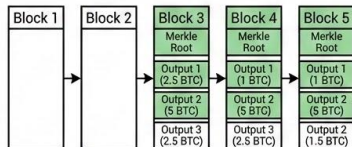


Storage and Scalability

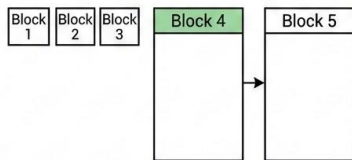
UTXO MODEL (FRAGMENTED OUTPUTS)



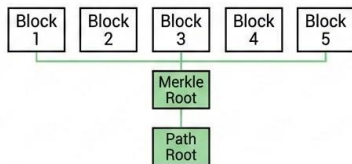
Node 1 (Full Node)



Node 2 (Pruned Node)



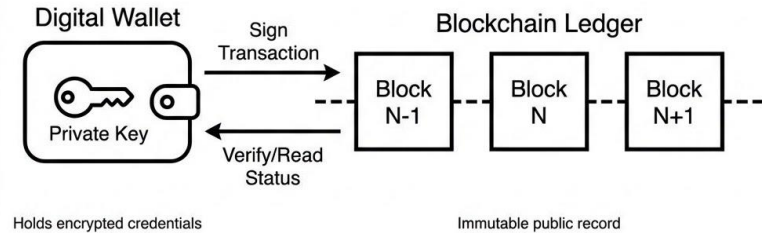
Node 3 (Light Node)



- Blockchain is append-only
- Data grows indefinitely
- Node types:
 - full node
 - pruned node
 - light client
- Tradeoff:
 - decentralization vs storage cost

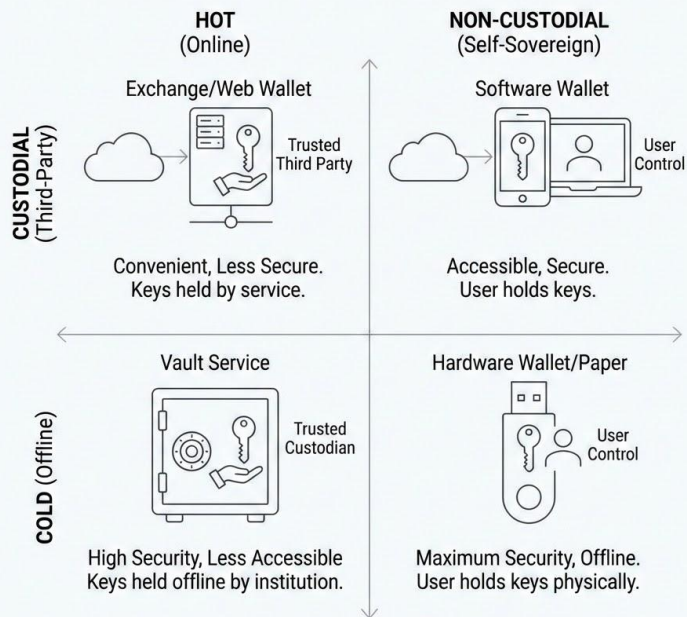
Wallets as Key Managers

- Wallet stores:
 - private keys
- Blockchain stores:
 - balances
- Ownership:
 - ability to sign transactions
- Functions:
 - generate keys
 - sign transactions
 - track addresses



Wallet Architectures

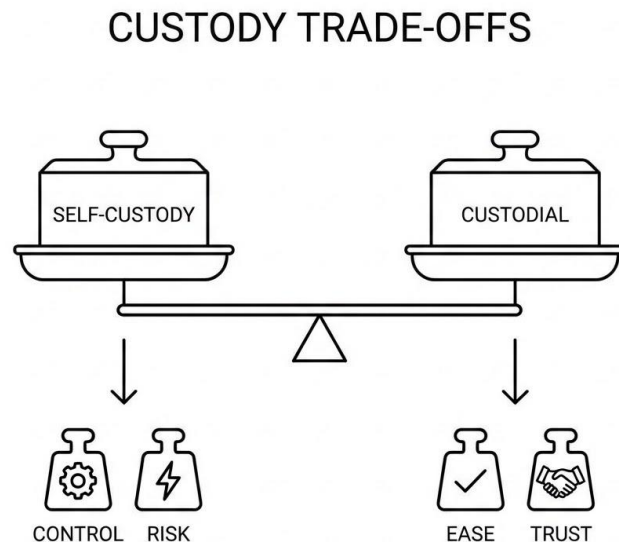
Cryptocurrency Wallet Comparison Matrix



- Hot wallets:
 - online, convenient
- Cold wallets:
 - offline, secure
- Custodial:
 - third party controls keys
- Non-custodial:
 - user controls keys
- Tradeoff:
 - usability vs security

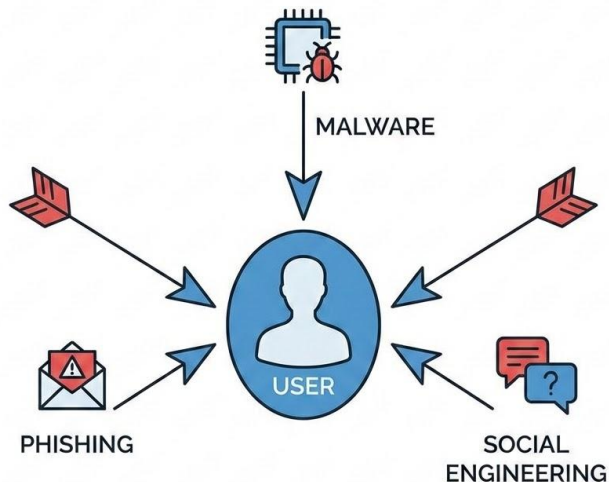
Custody: Control vs Responsibility

- **Self-custody:**
 - full control over assets
 - no recovery mechanism
 - user responsible for key protection
- **Custodial systems:**
 - third party manages keys
 - easier onboarding
 - introduces trust and counterparty risk
- **Real-world failures:**
 - exchange collapses
 - insider misuse
- **Key point:**
 - risk moves from protocol → user or institution



Where Systems Actually Break

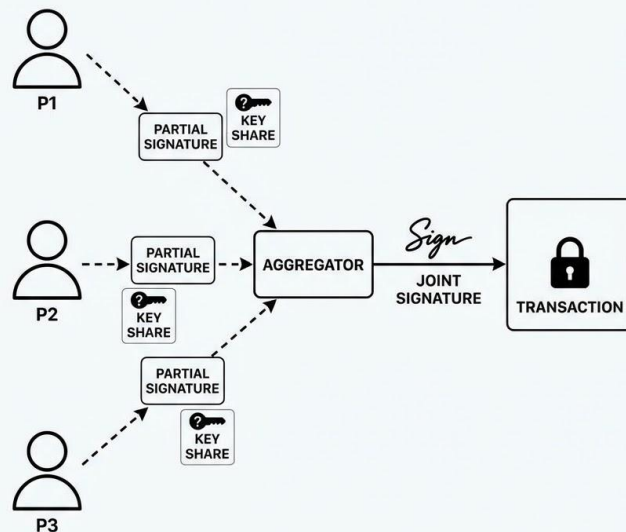
USER-CENTRIC ATTACK SURFACE



- Most attacks do NOT break cryptography
- Common vectors:
 - phishing (fake interfaces, fake wallets)
 - malware (key extraction)
 - social engineering
 - User is the weakest link
- Security must consider:
 - human behavior
 - interface design
 - operational practices
- Key insight:
 - system security $\neq$ cryptographic strength

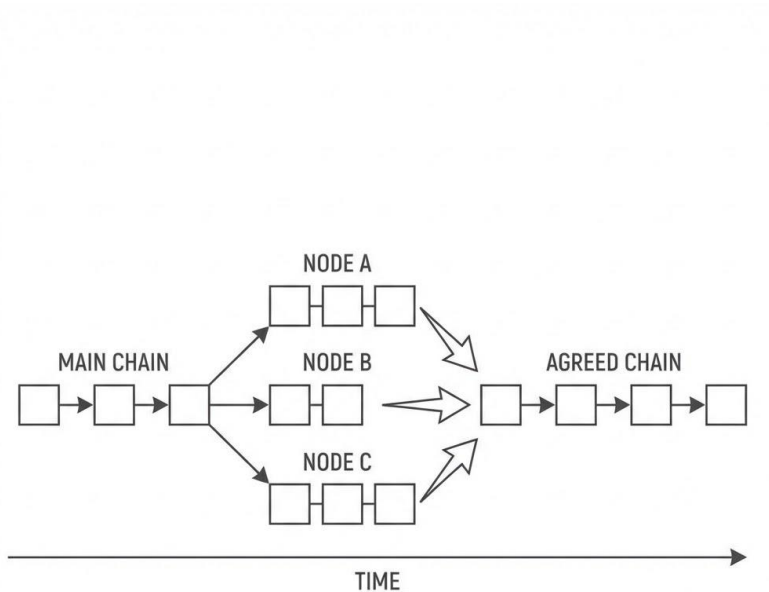
Distributed Key Control

- Multi-signature:
 - requires m-of-n keys
 - reduces single point of failure
- Threshold schemes:
 - flexible authorization policies
- MPC (Multi-Party Computation):
 - private key never reconstructed
 - signing distributed across parties
- Tradeoffs:
 - increased complexity
 - coordination overhead
- Key idea:
 - security becomes a protocol, not just a key



THRESHOLD SIGNATURE SCHEME: Participants contribute partial signatures to authorize a transaction without revealing individual private keys.

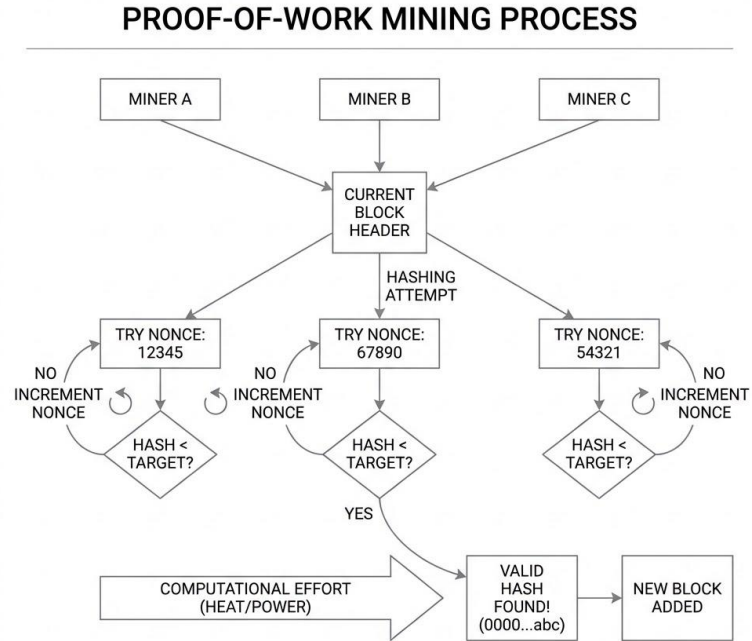
Agreement in a Distributed System



- Nodes operate independently
- Network delays → inconsistent views
- Conflicts:
 - different transaction orders
 - competing blocks
- Consensus provides:
 - Ordering
 - conflict resolution
 - shared state
- Goal:
 - converge to single history
- Key insight:
 - consistency is not given, it is enforced

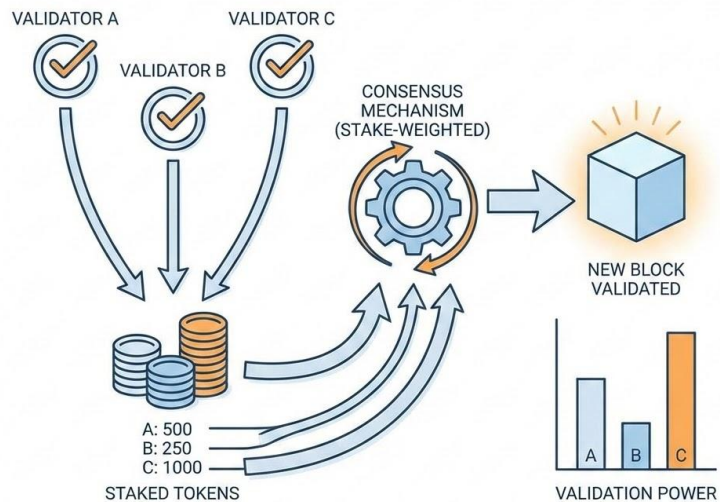
Proof of Work: Security via Computation

- Requires solving hash puzzle
- Mechanism:
 - find nonce such that $\text{hash} < \text{target}$
- Security:
 - attacker must control majority of compute power
- Properties:
 - probabilistic finality
 - high energy cost
- Strength:
 - simple and robust
- Key insight:
 - security = cost of rewriting history



Miners race to find a nonce that results in a hash below the network target, demonstrating computational work.

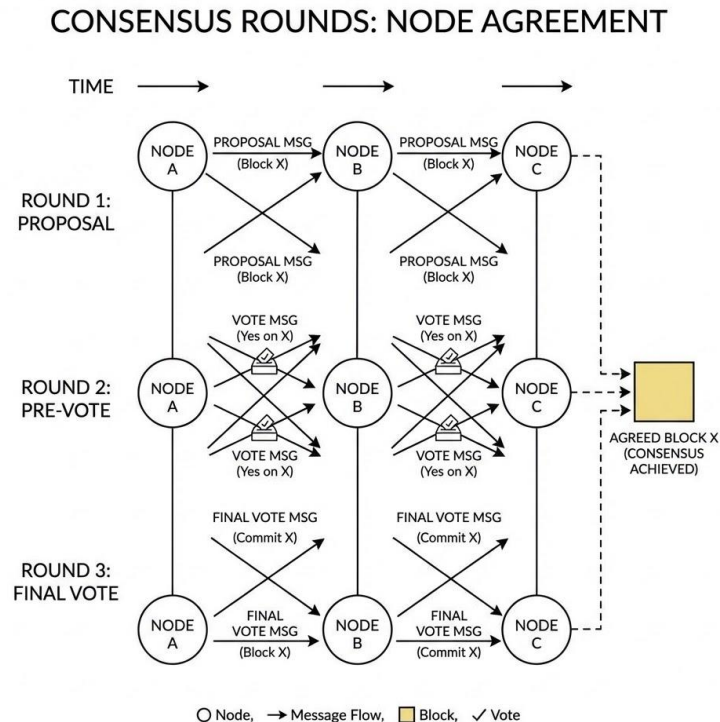
Proof of Stake: Security via Capital



- Validators lock tokens as stake
- Block selection proportional to stake
- Misbehavior:
 - penalized (slashing)
- Advantages:
 - lower energy usage
- Risks:
 - stake concentration
 - governance capture
- Requires:
 - strong economic incentives
- Key insight:
 - security tied to economic exposure

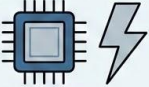
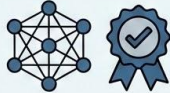
Byzantine Fault Tolerance (BFT)

- Nodes communicate and vote
- Assumes:
 - partially trusted participants
- Guarantees:
 - fast finality
 - deterministic agreement
- Requires:
 - known validator set
- Limitations:
 - scalability constraints
 - not permissionless
- Key insight:
 - replaces computation with coordination



Three Ways to Solve Sybil Resistance

CONSENSUS MECHANISMS COMPARISON: PoW, PoS, BFT

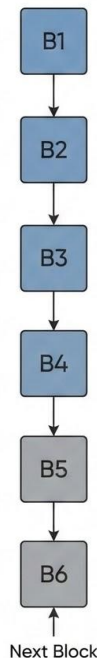
	PROOF-OF-WORK (PoW)	PROOF-OF-STAKE (PoS)	BYZANTINE FAULT TOLERANCE (BFT)
RESOURCE (Input)	 Computational Power (Electricity & Hardware)	 Staked Cryptocurrency (Economic Value)	 Reputation & Trust (Identity & Voting)
MECHANISM (Process)	 Miners compete to solve complex cryptographic puzzles; first to solve proposes block.	 Validators are selected to propose blocks based on the amount of stake they hold.	 A known set of validators vote in multiple rounds to achieve consensus (e.g., 2/3 majority).
RISKS (Trade-offs)	 High energy consumption, Centralization of mining pools, 51% attack.	 Nothing-at-Stake problem, Wealth concentration, Long-range attacks.	 Limited scalability (small number of nodes), Vulnerable if >1/3 nodes are malicious, Centralization risk in node selection.

- PoW:
 - resource = energy
 - attack = expensive computation
- PoS:
 - resource = capital
 - attack = financial exposure
- BFT:
 - resource = identity
 - attack = governance failure
- Tradeoffs:
 - Decentralization
 - Efficiency
 - security
- Key insight:
 - consensus defines who gets to decide

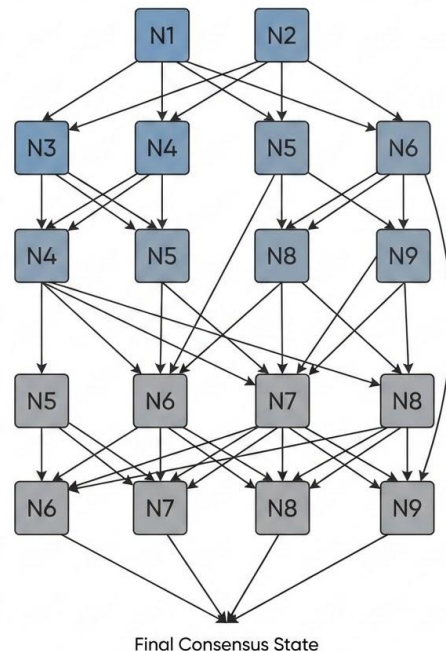
Alternative Ledger Structures

- **Blockchain:**
 - linear structure
- **DAG-based systems:**
 - parallel validation
- **Hashgraph:**
 - gossip + virtual voting
- **Motivation:**
 - Scalability
 - throughput
- **Observation:**
 - blockchain is one design choice
- **Key insight:**
 - structure defines performance limits

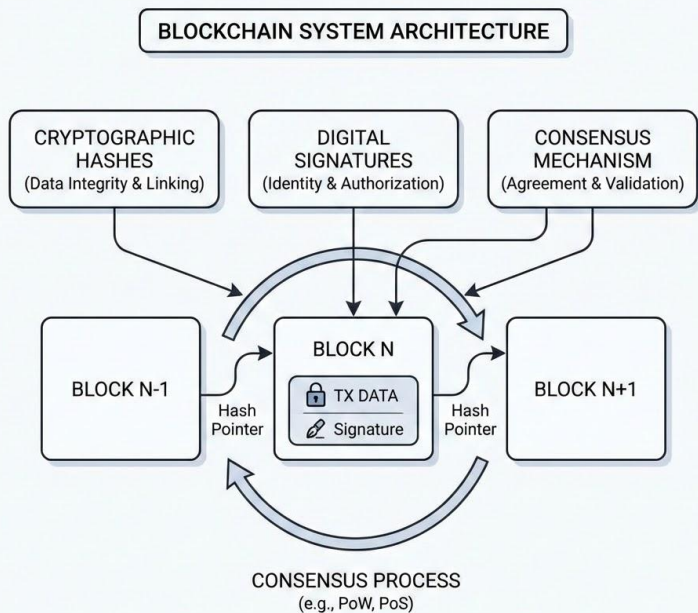
LINEAR BLOCKCHAIN
STRUCTURE



DIRECTED ACYCLIC GRAPH
(DAG) STRUCTURE



From Primitives to Systems



- Blockchain systems emerge from:
 - data structures
 - Cryptography
 - incentives
- Each layer solves a different problem:
 - integrity → hashes
 - ownership → signatures
 - agreement → consensus
- Security depends on:
 - correct composition
 - weakest component
- Key insight:
 - understanding primitives → understanding systems